MANUAL DE OPERAÇÃO DA PLATAFORMA DIGITAL

Do Repositório ao DOI: Como Criar, Publicar e Preservar seu Projeto no GitHub e no Zenodo

Série: Plataforma Amazônia Regenerativa — Infraestrutura Colaborativa de Conhecimento Aberto

Versão: 1.0 | **Data:** 28 de fevereiro de 2026

Tecnologia: Takwara | **Autor:** Fabio Resck Takwara

Licença: CC BY 4.0 — copie, distribua, adapte. Cite a fonte.

> *"Uma ideia que não pode ser encontrada não existe. Uma tecnologia que não pode ser replicada não se multiplica. Um conhecimento que não foi publicado pertence ao esquecimento."*

> *Este manual é o guia que eu gostaria de ter tido quando comecei. Ele foi escrito para quem nunca abriu um terminal, nunca ouviu falar de "commit" e acha que "repositório" é onde se guarda remédio. Se você tem um projeto que merece existir para além do seu HD, continue lendo.*

COMO USAR ESTE MANUAL

Este manual tem **três portas de entrada** — escolha a que corresponde à sua realidade hoje:

| Quem sou | Por onde começo |

| :-- | :-- |

| Nunca usei GitHub; quero publicar meu projeto do zero | Parte I → Seção 1 |

| Já tenho repositório e quero um DOI citável no Zenodo | Parte II → Seção 6 |

| Quero configurar colaboração e automação do projeto | Parte III → Seção 9 |

Você não precisa ler tudo de uma vez. Cada seção termina com **"Próximo passo concreto"** — uma ação que você pode tomar ainda hoje.

PARTE I — FUNDAMENTOS: O QUE É E POR QUE IMPORTA

O repositório que você ainda não tem — mas já precisava

---

1. A Lógica da Plataforma: Por Que GitHub + Zenodo?

1.1 O problema que este manual resolve

Projetos técnicos e científicos no Brasil morrem de um jeito muito específico: ficam em arquivos `.docx` no HD de quem os criou, em grupos de WhatsApp que ninguém mais encontra, ou em PDFs sem data e sem autoria rastreável. Quando o financiamento é aprovado, quando a universidade quer publicar, quando um parceiro internacional pede evidências — o projeto não tem endereço.

A combinação **GitHub + Zenodo** resolve esse problema de forma permanente e gratuita:

```
SEU PROJETO

↓

GITHUB (repositório)

├─ Versiona todos os documentos com data e autoria

├─ Permite colaboração de qualquer parte do mundo

├─ Publica automaticamente um portal web (GitHub Pages)

└─ Serve como "diário técnico" auditável do projeto

↓

ZENODO (arquivo científico)

├─ Gera um DOI permanente a cada versão publicada
```

```
├─ DOI é citável em artigos, relatórios e editais (BNDES, VERRA, COP)

├─ Preserva o projeto por no mínimo 20 anos (servidor CERN)

└─ Registra autoria com timestamp imutável
```

> 💡 **O DOI (Digital Object Identifier) é o "CPF" do seu documento científico.** Uma vez gerado, ele nunca muda e sempre aponta para o seu trabalho — mesmo que o link do GitHub mude.

1.2 O que cada ferramenta faz

| Ferramenta | Função principal | Custo | Quem usa igual |

| :-- | :-- | :-- | :-- |

| **GitHub** | Versionamento e colaboração de arquivos | Gratuito | NASA, INPE, Linux, Wikipedia |

| **VSCode** | Editor de texto e interface visual do GitHub | Gratuito | Desenvolvedores, pesquisadores, escritores |

| **Zenodo** | Arquivo científico com DOI permanente | Gratuito | CERN, VERRA, revistas como Nature e Science |

| **MkDocs** | Transforma seus arquivos `.md` num portal web | Gratuito | Documentação técnica de projetos globais |

1.3 O ecossistema da Plataforma Amazônia Regenerativa

Esta plataforma opera sobre três repositórios interligados:

```
REPO MÃE (origem e referência)

https://resck.github.io/Takwara-Tech/

└─ Autoria: Fabio Resck Takwara

└─ Licença: CC BY 4.0
```

```
└─ Tecnologia Takwara — documentação original


↕ (herda autoria e licença)

REPO DE PROJETOS (execução e colaboração)

https://github.com/takwaratec/projetos

└─ Projetos específicos: Amazônia Regenerativa, Biorrefinaria, SMGA

└─ Colaboradores: UnB/FUP, Filemon, Jesiel, parceiros regionais

└─ DOI Zenodo gerado a cada release


↕ (alimenta e é alimentado por)

PORTAL PÚBLICO (comunicação e disseminação)

https://takwaratec.github.io/projetos/

└─ MkDocs publicado via GitHub Pages

└─ Acessível a qualquer pessoa sem conta no GitHub

└─ Atualizado automaticamente a cada commit
```

Próximo passo concreto

> Abra o navegador e acesse github.com. Olhe a página inicial por 2 minutos sem fazer nada — observe que existem projetos de todo o mundo, de todos os tamanhos. O seu projeto merece estar aqui.

---

2. Criando sua Conta no GitHub

2.1 Passo a passo do cadastro

1. Acesse github.com/signup
2. Informe seu e-mail, crie uma senha e escolha um **nome de usuário** (username)

- 🎯 **Escolha bem:** o username aparece no endereço dos seus repositórios para sempre. Use algo profissional e identificável. Exemplos: `fabiotakwara`, `amazonia-regenerativa`, `takwaratec`

3. Complete a verificação de identidade e confirme seu e-mail
4. Escolha o plano **Free** (gratuito) — ele inclui repositórios públicos ilimitados, GitHub Pages e GitHub Actions sem custo

2.2 Configurações essenciais após o cadastro

Após criar a conta, configure três coisas antes de continuar:

A. Foto e perfil público

- Clique na foto no canto superior direito → **Settings** → **Public profile**
- Preencha: nome completo, bio (ex.: *"Inventor | Tecnologia Takwara | Bioeconomia Amazônica"*), localização e link do site (se tiver)
- Isso aparece em todos os seus projetos e identifica você como autor

B. Adicione uma chave SSH (recomendado, mas opcional no início)

- Para usuários iniciantes, o GitHub oferece autenticação via token HTTPS, que é suficiente
- A chave SSH é configurada pelo VSCode automaticamente na primeira vez que você conectar — o próprio editor vai perguntar

C. Confirme o e-mail

- Sem e-mail confirmado, você não pode criar repositórios públicos com GitHub Pages

Próximo passo concreto

> Crie sua conta agora. Leva 4 minutos. Guarde o **username** e a **senha** em algum lugar seguro (gerenciador de senhas ou papel físico numa gaveta). Você vai precisar deles toda vez que usar o VSCode.

---

3. Instalando o VSCode: Seu Centro de Controle

3.1 O que é o VSCode

O Visual Studio Code (VSCode) é um editor de texto gratuito da Microsoft que, além de editar arquivos, tem uma **interface visual completa para o GitHub** — você pode fazer commits, ver o que mudou, criar branches e publicar no GitHub sem precisar saber nenhum comando de terminal.

Para os fins deste manual, o VSCode é sua **janela para o repositório**: você edita os documentos do projeto aqui, e envia para o GitHub com cliques simples.

3.2 Instalação

1. Acesse code.visualstudio.com
2. Clique em **Download for Windows** (ou Mac/Linux)
3. Execute o instalador — mantenha todas as opções padrão marcadas, especialmente:

- ✅ *Add "Open with Code" action to Windows Explorer file context menu*
- ✅ *Register Code as an editor for supported file types*

4. Abra o VSCode após a instalação

3.3 Extensões essenciais para este projeto

Com o VSCode aberto, clique no ícone de quatro quadradinhos na barra lateral esquerda (**Extensions**, atalho `Ctrl+Shift+X`) e instale:

| Extensão | Para que serve | Como instalar |

| :-- | :-- | :-- |

| **GitLens** | Mostra quem editou o quê e quando, linha por linha | Buscar "GitLens" → Install |

| **Markdown All in One** | Pré-visualização e formatação de arquivos `.md` | Buscar "Markdown All in One" → Install |

| **GitHub Copilot** (opcional) | Sugestões de escrita por IA diretamente no editor | Buscar "GitHub Copilot" → Install (requer assinatura) |

3.4 Conectando o VSCode à sua conta GitHub

1. No VSCode, pressione `Ctrl+Shift+P` para abrir a **paleta de comandos**
2. Digite `GitHub: Sign in` e pressione Enter
3. O navegador vai abrir — faça login no GitHub e autorize o VSCode

4. Pronto: o VSCode está conectado à sua conta

Próximo passo concreto

> Instale o VSCode agora. Quando abrir pela primeira vez, ele vai sugerir um tema de cor — escolha o que preferir, não afeta nada técnico. Instale a extensão "Markdown All in One" e abra um arquivo `.md` qualquer do seu computador para ver a pré-visualização funcionando.

---

4. Criando seu Primeiro Repositório

4.1 O que é um repositório

Um **repositório** (ou "repo") é uma pasta inteligente que registra tudo que acontece dentro dela: quem criou cada arquivo, quando, qual era o conteúdo antes de cada mudança, e por quê foi alterado. É como um Google Docs com histórico infinito, mas para qualquer tipo de arquivo.

4.2 Criando o repositório no GitHub (via navegador)

1. Faça login em github.com
2. Clique no botão verde **New** (ou acesse github.com/new)
3. Preencha os campos:

| Campo | O que colocar | Exemplo |

| :-- | :-- | :-- |

| **Repository name** | Nome do seu projeto (sem espaços) | `amazonia-regenerativa` |

| **Description** | Uma frase descrevendo o projeto | `Plataforma de bioindústrias comunitárias de baixo carbono na Amazônia Legal` |

| **Visibility** | Public (para projetos abertos) ou Private | Public |

| **Add a README file** | ✅ Marcar | — |

| **Add .gitignore** | Pode deixar em branco por enquanto | — |

| **Choose a license** | **Creative Commons Zero v1.0 Universal** (depois você vai trocar para CC BY 4.0 manualmente) | — |

4. Clique em **Create repository**

> ⚠️ **Atenção ao nome:** use apenas letras minúsculas, números e hífens. Sem acentos, espaços ou caracteres especiais. O nome vira parte do endereço web do projeto: `github.com/seu-usuario/nome-do-repositorio`.

4.3 Estrutura de pastas recomendada para projetos da Plataforma

Antes de adicionar qualquer arquivo, pense na organização. A estrutura abaixo é o padrão da Plataforma Amazônia Regenerativa — adapte ao seu projeto:

```
meu-projeto/
│
├── README.md ← Capa do repositório (obrigatório)
├── AUTORIA.md ← Declaração de autoria e licença (obrigatório)
├── .zenodo.json ← Metadados para o DOI (veja Seção 6)
├── LICENSE ← Arquivo de licença CC BY 4.0
│
├── docs/ ← Documentos técnicos do projeto
│ ├── index.md ← Página inicial do portal web
│ ├── plataforma.md ← Documento principal
│ ├── memorial-tecnico.md ← Memorial descritivo
│ └── relatorio-executivo.md ← Relatório para financiadores
│
├── data/ ← Dados brutos (CSVs, shapefiles, tabelas)
│ └── inventario-bambu.csv
```

```
│
├── assets/ ← Imagens, logos, diagramas
│ └── logo-takwara.png
│
├── notebooks/ ← Scripts Python (para projetos com SMGA)
│ └── 01_gbif_coleta.py
│
└── mkdocs.yml ← Configuração do portal web (veja Seção 8)
```

Próximo passo concreto

> Crie o repositório agora pelo navegador. Depois de criado, copie o endereço da página (ex.: `https://github.com/seuusuario/meu-projeto`) — você vai precisar dele na próxima seção para clonar no VSCode.

---

5. Clonando, Editando e Publicando: O Fluxo Básico

5.1 O ciclo fundamental do GitHub

Todo trabalho no GitHub segue o mesmo ciclo de quatro etapas:

```
CLONE (uma vez só)
└─ Baixa o repositório do GitHub para o seu computador
```

```
↓

EDIT (quantas vezes quiser)

└─ Você edita os arquivos normalmente no VSCode

↓

COMMIT (registra a mudança)

└─ "Fotografa" o estado atual dos arquivos com uma mensagem explicando o que mudou

↓

PUSH (envia para o GitHub)

└─ Sobe o commit para o repositório online

↓

(volta para EDIT → próximo ciclo)
```

5.2 Clonando o repositório no VSCode

Passo 1: Copie o endereço do seu repositório no GitHub:

- Acesse a página do repositório
- Clique no botão verde **Code**
- Copie o endereço que aparece (começa com `https://github.com/...`)

Passo 2: No VSCode:

- Pressione `Ctrl+Shift+P`
- Digite `Git: Clone` e pressione Enter
- Cole o endereço copiado

- Escolha uma pasta no seu computador para salvar (ex.: `Documentos/projetos/`)
- Clique em **Open** quando o VSCode perguntar se quer abrir a pasta clonada

Agora você tem uma cópia local do repositório. Tudo que você editar aqui pode ser enviado de volta para o GitHub.

5.3 Adicionando arquivos ao repositório

Opção A — Arrastar e soltar (para iniciantes):

- No explorador de arquivos do Windows, selecione os documentos que quer adicionar
- Arraste para dentro da pasta do projeto no VSCode (painel lateral esquerdo)
- Os arquivos aparecerão com uma letra **U** (Untracked = ainda não rastreados)

Opção B — Copiar diretamente:

- Abra a pasta do projeto no explorador de arquivos (`Ctrl+Shift+E` → clique com botão direito → *Reveal in File Explorer*)
- Cole os arquivos diretamente na pasta

5.4 Fazendo um Commit: registrando a mudança

1. No VSCode, clique no ícone de **ramificação** na barra lateral esquerda (`Ctrl+Shift+G`) — é o painel **Source Control**
2. Você verá seus arquivos listados em **Changes**
3. Para cada arquivo que quer incluir neste commit:

- Passe o mouse sobre o nome do arquivo
- Clique no `+` → o arquivo vai para **Staged Changes**

4. Na caixa de texto no topo, escreva uma mensagem descritiva:

```
✅ Mensagens boas:

"docs: adiciona Plataforma Amazônia Regenerativa v5.1"

"feat: inclui memorial técnico da biorrefinaria"
```

```
"fix: corrige referências bibliográficas na Seção 10"



❌ Mensagens ruins:

"update"

"arquivo novo"

"mudanças"
```

5. Clique no botão ✓ **Commit** (ou `Ctrl+Enter`)

> 💡 **Por que a mensagem importa?** Cada commit fica registrado para sempre no histórico do projeto. Daqui a 5 anos, você (ou um auditor do BNDES, ou um pesquisador da UFAC) vai poder ver exatamente o que mudou em cada versão. Mensagens claras são sua memória técnica.

5.5 Enviando para o GitHub (Push)

Após o commit, os arquivos ainda estão só no seu computador. Para enviá-los ao GitHub:

1. No painel **Source Control**, clique em `...` (menu de ações)
2. Clique em **Push**
3. Se for a primeira vez, o VSCode vai pedir suas credenciais do GitHub — faça login
4. Aguarde a barra de progresso no rodapé. Quando desaparecer, está feito.

Verificação: acesse `github.com/seuusuario/meu-repositorio` no navegador — seus arquivos devem aparecer lá.

5.6 Verificando o que mudou: Git Status

Antes de qualquer commit, é uma boa prática verificar o estado atual. Abra o terminal integrado (`Ctrl+` `) e execute:

```
git status
```

O resultado mostra:

- **Untracked files** = arquivos novos que o Git ainda não conhece
- **Modified** = arquivos já rastreados que foram alterados
- **Staged** = arquivos prontos para o próximo commit

```
# Outros comandos úteis para iniciantes:

git log --oneline # mostra o histórico de commits em uma linha cada

git diff # mostra exatamente o que mudou em cada arquivo
```

Próximo passo concreto

> Adicione o arquivo `AUTORIA.md` ao repositório (veja o modelo na caixa abaixo), faça o Stage, escreva a mensagem de commit `"docs: adiciona declaração de autoria e licença CC BY 4.0"` e dê Push. É o commit mais importante do projeto.

📝 MODELO DE ARQUIVO `AUTORIA.md`

(Coloque na pasta raiz do repositório. Adapte os campos em CAIXA ALTA.)

```
# Autoria e Licenciamento



**Autor original:** FABIO RESCK TAKWARA

**Repositório de origem:** https://resck.github.io/Takwara-Tech/

**Repositório do projeto:** https://github.com/SEUUSUARIO/SEU-REPOSITORIO

**Data de criação original:** [MÊS/ANO DO PRIMEIRO DOCUMENTO]
```

Licença: Creative Commons Attribution 4.0 International (CC BY 4.0)

***

O que esta licença permite

- ✅ **Compartilhar** — copiar e redistribuir em qualquer meio ou formato
- ✅ **Adaptar** — remixar, transformar e criar a partir do material
- ✅ **Para qualquer fim** — inclusive comercial

O que esta licença exige

- 📌 **Atribuição** — você deve dar o crédito adequado ao autor original,

fornecer um link para a licença e indicar se foram feitas alterações.

Forma correta de citar este trabalho

> TAKWARA, Fabio Resck. *[NOME DO DOCUMENTO]*. Plataforma Amazônia

> Regenerativa, [ANO]. Disponível em: [URL DO REPOSITÓRIO].

> DOI: [DOI DO ZENODO — preencher após a primeira release].

> Licença CC BY 4.0.

***

Sobre a parceria institucional

Este repositório pode ter [NOME DA INSTITUIÇÃO PARCEIRA, ex.: UnB/FUP]
como proponente institucional para fins de submissão a editais de
financiamento (BNDES, Fundo Amazônia, etc.).

Essa parceria não implica transferência de titularidade intelectual
sobre a tecnologia original nem sobre os documentos aqui publicados.
A autoria original permanece com Fabio Resck Takwara, conforme
declarado no Repo Mãe e registrado no Zenodo.

***

Texto completo da licença: creativecommons.org/licenses/by/4.0

PARTE II — DO REPOSITÓRIO AO DOI: PUBLICANDO COM O ZENODO

Tornando seu projeto citável e permanente

---

6. Configurando a Autoria com o Arquivo `.zenodo.json`

6.1 O que é e por que criar este arquivo

O arquivo `.zenodo.json` fica na raiz do repositório e instrui o Zenodo sobre **quem são os autores, qual é o título, qual é a licença e como categorizar o trabalho** quando um DOI for gerado. Sem ele, o Zenodo usa informações genéricas do GitHub, que raramente estão corretas.

6.2 Criando o arquivo no VSCode

No VSCode, crie um novo arquivo chamado exatamente `.zenodo.json` (com o ponto no início) na pasta raiz do repositório e cole o conteúdo abaixo, adaptando os campos:

```
{

"title": "Plataforma Amazônia Regenerativa — Anais Técnicos e Documentação de Projeto",

"description": "Documentação técnica e científica da Plataforma Amazônia Regenerativa: biorrefinaria modular de bambu Guadua spp.,
saneamento ecológico, biocompósitos, Sistema de Monitoramento Geoespacial Automatizado (SMGA) e bioeconomia comunitária de baixo
carbono na Amazônia Legal.",

"upload_type": "publication",

"publication_type": "technicalnote",

"license": "CC-BY-4.0",

"access_right": "open",

"language": "por",

"creators": [
```

```
{
"name": "Takwara, Fabio Resck",
"affiliation": "Tecnologia Takwara — Plataforma Amazônia Regenerativa",
"orcid": "COLOQUE-SEU-ORCID-AQUI-OU-REMOVA-ESTA-LINHA"
}
],
"contributors": [
{
"name": "Cruz, Tania Cristina",
"affiliation": "Universidade de Brasília — FUP/CDT",
"type": "ProjectMember"
},
{
"name": "Tiago, Filemon",
"affiliation": "Arquitetura BioRrevolucionária",
"type": "ProjectMember"
}
],
"keywords": [
"bambu",
```

```
"bioeconomia",
"Amazônia",
"biorrefinaria",
"biochar",
"saneamento ecológico",
"habitação social",
"tecnologia social",
"ciência aberta",
"CC BY 4.0"
],
"grants": [],
"related_identifiers": [
{
"identifier": "https://resck.github.io/Takwara-Tech/",
"relation": "isPartOf",
"resource_type": "other"
}
]
}
```

> 💡 **ORCID:** é o "CPF dos pesquisadores" — um identificador permanente que conecta sua produção científica à sua identidade. É gratuito e leva 5 minutos para criar em orcid.org. Fortemente recomendado se você vai publicar artigos ou submeter projetos a financiadores internacionais.

6.3 Adicionando o arquivo de licença

Na raiz do repositório, crie também o arquivo `LICENSE` (sem extensão) com o seguinte conteúdo:

```
Creative Commons Attribution 4.0 International (CC BY 4.0)


Copyright (c) 2026 Fabio Resck Takwara


You are free to:

Share — copy and redistribute the material in any medium or format

Adapt — remix, transform, and build upon the material for any purpose,

even commercially.


Under the following terms:

Attribution — You must give appropriate credit, provide a link to the license,

and indicate if changes were made.


Full license text: https://creativecommons.org/licenses/by/4.0/legalcode
```

Próximo passo concreto

> Crie o `.zenodo.json` e o `LICENSE` no VSCode, faça Stage de ambos, mensagem de commit: `"chore: adiciona metadados Zenodo e licença CC BY 4.0"`, e dê Push.

---

7. Criando sua Conta no Zenodo e Conectando ao GitHub

7.1 Cadastro no Zenodo

1. Acesse zenodo.org
2. Clique em **Log in** → **Log in with GitHub**
3. Autorize o Zenodo a acessar sua conta GitHub quando solicitado
4. Pronto — sua conta Zenodo está criada e conectada ao GitHub

> 🏛️ **O que é o Zenodo?** É o repositório científico aberto do CERN (o laboratório de física de partículas que inventou a internet). Documentos publicados aqui são preservados por no mínimo 20 anos, são indexados pelo Google Scholar, e seus DOIs são aceitos por revistas científicas como *Nature*, *Science* e pelo Fundo Amazônia.

7.2 Ativando o repositório no Zenodo

1. No Zenodo, vá em seu nome de usuário (canto superior direito) → **GitHub**
2. Você verá uma lista de todos os seus repositórios GitHub
3. Encontre o repositório que quer conectar e **ative o toggle** (botão deslizante) ao lado dele
4. Confirme se solicitado

A partir de agora, **toda vez que você criar uma Release no GitHub, o Zenodo automaticamente arquiva aquela versão e gera um DOI**.

7.3 O que é uma Release e como criar

Uma **Release** é uma "foto oficial" do repositório em um determinado momento — como uma edição de um livro. O DOI do Zenodo é gerado para cada Release.

Para criar sua primeira Release:

1. Na página do repositório no GitHub, clique em **Releases** (menu lateral direito)
2. Clique em **Create a new release**
3. Preencha os campos:

| Campo | O que colocar | Exemplo |

| :-- | :-- | :-- |

| **Tag version** | Número de versão semântico | `v1.0.0` |

| **Release title** | Nome desta versão | `Plataforma Amazônia Regenerativa — Versão Inicial` |

| **Description** | O que está incluído nesta versão | Lista dos documentos incluídos, data, contexto |

4. Clique em **Publish release**
5. Aguarde alguns minutos e acesse o Zenodo — o DOI já estará disponível na página do repositório

7.4 Estrutura dos números de versão (versionamento semântico)

```
v MAJOR . MINOR . PATCH

↓ ↓ ↓

1 . 0 . 0



MAJOR: mudança significativa no projeto (nova fase, reformulação)

MINOR: adição de novos documentos ou seções

PATCH: correções, typos, pequenas atualizações
```

Exemplos:

- `v1.0.0` → Primeira versão pública oficial
- `v1.1.0` → Adicionou o Memorial Técnico da Biorrefinaria
- `v1.1.1` → Corrigiu referência bibliográfica
- `v2.0.0` → Atualização maior (ex.: inclusão do componente de saneamento)

7.5 Localizando e usando o DOI

Após a Release, o Zenodo gera dois tipos de DOI:

| Tipo | O que aponta | Quando usar |

| :-- | :-- | :-- |

| **DOI de versão específica** | Exatamente aquela Release (v1.0.0) | Quando citar um documento que não deve mudar |

| **DOI conceito** | Sempre a versão mais recente | Para o README e links permanentes |

Adicione o DOI no seu README.md:

```
[
```

Substitua `XXXXXXX` pelo número do seu DOI. Isso gera um **badge clicável** na capa do repositório — um sinal visual de que o projeto é citável e verificável.

Próximo passo concreto

> Crie sua conta no Zenodo via GitHub, ative o repositório e crie a primeira Release agora. O DOI vai ser gerado em minutos. Copie o DOI e cole no arquivo `AUTORIA.md` no campo indicado. Faça um novo commit: `"docs: adiciona DOI Zenodo v1.0.0 ao arquivo de autoria"`.

---

8. Publicando um Portal Web com GitHub Pages e MkDocs

8.1 O que é GitHub Pages

O **GitHub Pages** transforma automaticamente os arquivos `.md` do seu repositório em um site público acessível por qualquer pessoa — sem precisar de hospedagem paga, sem precisar saber HTML.

O endereço segue o padrão:

`https://SEUUSUARIO.github.io/REPOSITORIO/`

8.2 Ativando o GitHub Pages

Opção A — Páginas simples (sem MkDocs):

1. Vá em **Settings** do repositório → **Pages** (menu lateral)
2. Em **Source**, selecione **Deploy from a branch**
3. Selecione a branch `main` e a pasta `/docs`
4. Clique em **Save**
5. Em poucos minutos, o site estará no ar

Opção B — Portal completo com MkDocs (recomendado):

Crie o arquivo `mkdocs.yml` na raiz do repositório:

```yaml
site_name: Plataforma Amazônia Regenerativa

site_description: Documentação técnica e científica da Plataforma

site_author: Fabio Resck Takwara

site_url: https://takwaratec.github.io/projetos/


repo_name: takwaratec/projetos

repo_url: https://github.com/takwaratec/projetos

edit_uri: edit/main/docs/


theme:

name: material

language: pt

palette:

primary: green
```

```yaml
accent: amber

features:

- navigation.tabs

- navigation.sections

- search.highlight



nav:

- Início: index.md

- Plataforma: plataforma.md

- Memorial Técnico: memorial-tecnico.md

- Relatório Executivo: relatorio-executivo.md

- Autoria: ../AUTORIA.md



extra:

social:

- icon: fontawesome/brands/github

link: https://github.com/takwaratec/projetos



copyright: "CC BY 4.0 | Fabio Resck Takwara | Plataforma Amazônia Regenerativa"
```

Este arquivo instrui o MkDocs a gerar um site completo com menu de navegação, busca e tema visual verde — publicado automaticamente via GitHub Actions a cada novo commit.

Próximo passo concreto

> Ative o GitHub Pages pelo painel de Settings. Depois de alguns minutos, acesse o endereço gerado — você terá um portal público do projeto que pode ser compartilhado com financiadores, parceiros e pesquisadores.

PARTE III — COLABORAÇÃO, AUTOMAÇÃO E BOAS PRÁTICAS

Transformando o repositório em infraestrutura viva de ciência aberta

---

9. Trabalhando em Equipe: Issues, Pull Requests e Permissões

9.1 Como outros colaboradores acessam o repositório

No GitHub, existem duas formas de colaborar:

| Tipo | Quem usa | Como funciona |

| :-- | :-- | :-- |

| **Colaborador direto** | Equipe de confiança (Filemon, Jesiel, bolsistas) | Você adiciona como colaborador em Settings → Collaborators; eles podem editar diretamente |

| **Fork + Pull Request** | Pesquisadores externos, parceiros da UFAC, comunidade | Eles fazem uma cópia do repositório, propõem mudanças, e você revisa antes de aceitar |

Adicionando um colaborador:

1. Vá em **Settings** → **Collaborators** → **Add people**
2. Busque pelo username ou e-mail do GitHub da pessoa
3. Defina o nível de acesso: `Write` (pode editar) ou `Triage` (pode comentar, não edita)

9.2 Issues: o sistema de tarefas do projeto

Uma **Issue** é uma tarefa, dúvida, sugestão ou problema registrado publicamente no repositório. É o equivalente a um "Trello" integrado ao projeto.

Criando uma Issue:

1. Na página do repositório, clique em **Issues** → **New issue**
2. Dê um título claro: ex. *"Adicionar dados GBIF da Guadua no Acre"*
3. Descreva o que precisa ser feito
4. Atribua a um colaborador em **Assignees** (opcional)
5. Adicione um **label** (etiqueta): `documentation`, `research`, `enhancement`

Por que usar Issues mesmo trabalhando sozinho?

Porque quando o BNDES ou o Fundo Amazônia analisarem o projeto, eles vão ver um histórico de trabalho organizado, com problemas identificados e resolvidos — evidência de gestão de qualidade.

9.3 O arquivo README.md: a capa do projeto

O `README.md` é a primeira coisa que qualquer visitante do repositório vê. Ele deve responder em 30 segundos:

1. O que é este projeto?
2. Quem fez?
3. Como usar/contribuir?
4. Onde está publicado?

Template básico para o README.md:

```
# 🌿 Plataforma Amazônia Regenerativa



[

[



**Polo integrado de bioindústrias comunitárias de baixo carbono na Amazônia Legal.**
```

```
***

## Sobre o Projeto

A Plataforma Amazônia Regenerativa articula manejo ecológico de bambu
*Guadua* spp., biorrefinaria modular, saneamento ecológico e biocompósitos
para habitação social resiliente no Acre.

**Autor original:** Fabio Resck Takwara — [Tecnologia Takwara](https://resck.github.io/Takwara-Tech/)
**Parceria institucional:** UnB/FUP — Profa. Dra. Tania Cristina Cruz
**Licença:** CC BY 4.0

***

## Documentos Principais

| Documento | Descrição | Link |
| :-- | :-- | :-- |
| Plataforma v5.1 | Documento técnico completo | [docs/plataforma.md](docs/plataforma.md) |
```

```
| Memorial Técnico | Biorrefinaria modular de bambu | [docs/memorial-tecnico.md](docs/memorial-tecnico.md) |

| Relatório Executivo | Para financiadores e comitês de investimento | [docs/relatorio-executivo.md](docs/relatorio-executivo.md) |


***


## Como Contribuir


1. Abra uma [Issue](https://github.com/SEUUSUARIO/REPOSITORIO/issues) descrevendo sua contribuição
2. Faça um Fork do repositório
3. Crie um Pull Request com suas alterações
4. Aguarde revisão


***


## Como Citar


> TAKWARA, Fabio Resck. *Plataforma Amazônia Regenerativa — Anais Técnicos*.
> v1.0.0. 2026. DOI: [10.5281/zenodo.XXXXXXX](https://doi.org/10.5281/zenodo.XXXXXXX).
> Licença CC BY 4.0.
```

```
***


*Portal público: [takwaratec.github.io/projetos](https://takwaratec.github.io/projetos/)*
```

---

10. Automação com GitHub Actions: O Agente que Trabalha por Você

10.1 O que é GitHub Actions

O **GitHub Actions** é o sistema de automação do GitHub. Você escreve um arquivo de instrução (`.yml`) e o GitHub executa ações automaticamente quando algo acontece no repositório — por exemplo, toda segunda-feira, ou toda vez que você faz um commit.

Para a Plataforma Amazônia Regenerativa, o GitHub Actions é o **agente coordenador** que executa o pipeline SMGA (Sistema de Monitoramento Geoespacial Automatizado), coleta dados do GBIF, gera relatórios e mantém o portal MkDocs atualizado.

10.2 Criando o primeiro workflow: publicação automática do MkDocs

Crie o arquivo `.github/workflows/deploy-docs.yml` no repositório:

```
name: Publicar Portal MkDocs



on:

push:

branches:

- main # Executa sempre que há commit na branch main
```

```yaml
jobs:

deploy:

runs-on: ubuntu-latest

steps:

- name: Clonar repositório

uses: actions/checkout@v4



- name: Configurar Python

uses: actions/setup-python@v4

with:

python-version: '3.11'



- name: Instalar MkDocs

run: pip install mkdocs-material



- name: Publicar no GitHub Pages

run: mkdocs gh-deploy --force
```

O que este arquivo faz:

1. Toda vez que você faz um Push para a branch `main`...
2. O GitHub cria uma máquina virtual temporária (Linux)...

3. Instala o MkDocs...
4. Constrói o site HTML a partir dos seus arquivos `.md`...
5. Publica automaticamente no GitHub Pages

Resultado: **seu portal web é atualizado automaticamente toda vez que você edita um documento e faz Push**. Você nunca precisa publicar manualmente.

10.3 Criando o workflow do SMGA (para projetos com dados geoespaciais)

Para projetos que incluem o Sistema de Monitoramento Geoespacial, crie `.github/workflows/smga-pipeline.yml`:

```yaml
name: SMGA — Pipeline Semanal de Monitoramento


on:

schedule:

- cron: '0 6 * * 1' # Toda segunda-feira às 06h UTC (03h Brasília)

workflow_dispatch: # Permite execução manual quando necessário


jobs:

coletar-dados:

runs-on: ubuntu-latest

steps:

- name: Clonar repositório

uses: actions/checkout@v4
```

```yaml
- name: Configurar Python
uses: actions/setup-python@v4
with:
python-version: '3.11'

- name: Instalar dependências
run: |
pip install pygbif pandas geopandas

- name: Executar coleta GBIF
run: python notebooks/01_gbif_coleta.py

- name: Commitar dados atualizados
run: |
git config --global user.email "smga@amazonia-regenerativa.org"
git config --global user.name "SMGA Bot"
git add data/
git diff --staged --quiet || git commit -m "SMGA: atualização automática $(date +'%Y-%m-%d')"
git push
```

O que este agente faz:

- Toda segunda-feira de manhã, automaticamente...
- Coleta os registros mais recentes de Guadua do GBIF...
- Salva os dados na pasta `data/` ...
- Faz um commit automático com a data...
- Garante que o repositório sempre tem dados frescos, auditáveis e rastreáveis

---

11. Referência Rápida: Comandos do Terminal Mais Usados

Para quem quiser ir além da interface visual do VSCode, estes são os comandos básicos do Git que você vai usar no dia a dia:

```
# VERIFICAÇÃO

git status # Mostra o que mudou

git log --oneline # Histórico de commits resumido

git diff # Mostra as diferenças linha a linha



# CICLO BÁSICO

git add ARQUIVO.md # Adiciona um arquivo ao Stage

git add . # Adiciona TODOS os arquivos alterados

git commit -m "mensagem" # Registra o commit com mensagem

git push # Envia para o GitHub



# ATUALIZAÇÃO
```

```
git pull # Baixa as últimas alterações do GitHub



# SITUAÇÕES ESPECIAIS

git restore ARQUIVO.md # Desfaz alterações não commitadas

git log --oneline -10 # Mostra os últimos 10 commits
```

> ⚠️ **Regra de ouro para iniciantes: Nunca** execute `git reset --hard` ou `git force push` sem entender o que está fazendo. Esses comandos podem apagar commits permanentemente.

PARTE IV — GLOSSÁRIO E REFERÊNCIAS

Para quem quer entender cada termo

---

12. Glossário Técnico

| Termo | Definição em linguagem simples |

| :-- | :-- |

| **Repositório** | Pasta inteligente que guarda o histórico completo de todos os arquivos |

| **Commit** | "Fotografia" do estado do repositório em um momento, com mensagem explicativa |

| **Branch** | Versão paralela do repositório para testar mudanças sem afetar o principal |

| **Push** | Enviar commits do computador para o GitHub (internet) |

| **Pull** | Baixar as atualizações do GitHub para o computador |

| **Clone** | Fazer uma cópia local de um repositório remoto (pela primeira vez) |

| **Fork** | Criar uma cópia de um repositório em outra conta (para contribuições externas) |

| **Pull Request** | Proposta de mudança enviada por um colaborador externo para revisão |

| **Issue** | Registro de tarefa, dúvida ou sugestão no projeto |

| **Release** | Versão oficial do projeto em um momento específico (gera o DOI) |

| **DOI** | Digital Object Identifier — identificador permanente e citável de um documento |

| **GitHub Pages** | Serviço que transforma arquivos `.md` do repositório em site público |

| **GitHub Actions** | Sistema de automação que executa scripts automaticamente |

| **MkDocs** | Ferramenta que transforma arquivos `.md` em portal web navegável |

| **Zenodo** | Repositório científico do CERN que arquiva projetos e gera DOIs |

| **CC BY 4.0** | Licença Creative Commons que permite uso livre com obrigação de citar o autor |

| **ORCID** | Identificador permanente de pesquisador, equivalente ao "CPF científico" |

| **`.zenodo.json`** | Arquivo com metadados de autoria lidos pelo Zenodo ao gerar o DOI |

| **README.md** | Arquivo de apresentação na capa do repositório |

| **AUTORIA.md** | Declaração explícita de autoria e licença (padrão da Plataforma Takwara) |

| **Staged Changes** | Arquivos selecionados para entrar no próximo commit |

| **Untracked** | Arquivos ainda não rastreados pelo Git |

---

13. Fontes, Links e Referências Técnicas

Documentação oficial das ferramentas:

| Ferramenta | Documentação | Link |

| :-- | :-- | :-- |

| GitHub — Primeiros passos | Guia oficial para iniciantes | docs.github.com/pt |

| GitHub Pages | Como publicar sites | pages.github.com |

| GitHub Actions | Automação e workflows | docs.github.com/actions |

| VSCode + Git | Controle de versão no editor | code.visualstudio.com/docs/sourcecontrol |

| Zenodo | Arquivo científico e DOIs | help.zenodo.org |

| Zenodo + GitHub | Integração passo a passo | docs.github.com/repositories/archiving |

| MkDocs Material | Tema e configuração | squidfunk.github.io/mkdocs-material |

Criação de identificadores científicos:

| Serviço | O que fornece | Link |

| :-- | :-- | :-- |

| ORCID | Identificador permanente de pesquisador | orcid.org |

| Zenodo | DOI para repositórios, datasets e documentos | zenodo.org |

| OSF | Pré-registro e arquivo de projetos de pesquisa | osf.io |

Sobre licenças abertas:

| Recurso | Link |

| :-- | :-- |

| Creative Commons CC BY 4.0 — texto completo | creativecommons.org/licenses/by/4.0 |

| Escolhendo uma licença — guia interativo | choosealicense.com |

---

14. Checklist de Lançamento: Tudo que Precisa Estar Feito

Antes de divulgar o repositório a financiadores, parceiros institucionais ou mídia, verifique:

INFRAESTRUTURA BÁSICA

[ ] Conta GitHub criada com username profissional

[ ] VSCode instalado e conectado ao GitHub

[ ] Repositório criado como Public

[ ] Estrutura de pastas organizada (docs/, data/, assets/)

AUTORIA E LICENÇA

[ ] Arquivo AUTORIA.md criado e commitado

[ ] Arquivo LICENSE (CC BY 4.0) criado e commitado

[ ] Arquivo .zenodo.json criado com autoria correta

CONTEÚDO

[ ] README.md com descrição do projeto, autoria e instruções de citação

[ ] Documentos principais na pasta docs/

[ ] Pelo menos um commit com mensagem descritiva

PUBLICAÇÃO

[ ] GitHub Pages ativo (Settings → Pages)

[ ] Portal web acessível em github.io/...

```
[ ] Conta Zenodo criada via GitHub

[ ] Repositório ativado no Zenodo

[ ] Primeira Release criada (v1.0.0)

[ ] DOI gerado e anotado no README.md e AUTORIA.md

[ ] Badge DOI aparecendo na capa do repositório


COLABORAÇÃO

[ ] Issues abertas com tarefas prioritárias do projeto

[ ] Colaboradores adicionados (se aplicável)

[ ] Workflow de publicação automática do MkDocs configurado
```

---

*Este manual integra a Série de Documentação Operacional da Plataforma Amazônia Regenerativa.

Documento vivo: atualizado a cada nova versão do repositório.*

Licença CC BY 4.0.

🎋 **Takwara — Tecnologia do Bambu para a Soberania Amazônica**

Repositório Mãe: resck.github.io/Takwara-Tech

DOI: *(a ser preenchido após a primeira Release)*

***